```
  GNU nano 8.7.1                 memory_alloc.c *
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n = 5;
    int *arr_malloc, *arr_calloc, *arr_realloc;

    printf("=== 1. malloc() demonstration ===\n");
    // malloc allocates uninitialized memory
    arr_malloc = (int *)malloc(n * sizeof(int));
    if (arr_malloc == NULL) {
        perror("malloc failed");
        exit(1);
    }
    for (int i = 0; i < n; i++) {
        arr_malloc[i] = (i + 1) * 10;
        printf("arr_malloc[%d] = %d\n", i, arr_malloc[i]);
    }

    printf("\n=== 2. calloc() demonstration ===\n");
    // calloc allocates memory and initializes all bytes to ze>
    arr_calloc = (int *)calloc(n, sizeof(int));
    if (arr_calloc == NULL) {
        perror("calloc failed");
        free(arr_malloc);
        exit(1);
    }
    for (int i = 0; i < n; i++) {
        printf("arr_calloc[%d] = %d (initialized to zero)\n", >
    }

    printf("\n=== 3. realloc() demonstration ===\n");
    // Resize arr_malloc to hold 10 integers instead of 5
```

```c
    // Resize arr_malloc to hold 10 integers instead of 5
    int new_size = 10;
    arr_realloc = (int *)realloc(arr_malloc, new_size * sizeof>
    if (arr_realloc == NULL) {
        perror("realloc failed");
        free(arr_calloc);
        exit(1);
    }
    arr_malloc = arr_realloc; // Update pointer

    for (int i = 5; i < new_size; i++) {
        arr_malloc[i] = (i + 1) * 10;
    }

    printf("After realloc (size 10):\n");
    for (int i = 0; i < new_size; i++) {
        printf("arr_malloc[%d] = %d\n", i, arr_malloc[i]);
    }

    printf("\n=== 4. free() demonstration (Intentional Leak fo>
    // Freeing arr_malloc and arr_calloc properly
    free(arr_malloc);
    free(arr_calloc);

    // NOTE: Intentionally NOT freeing a small block to demons>
    int *leaked_memory = (int *)malloc(100 * sizeof(int));
    leaked_memory[0] = 999;
    // free(leaked_memory); -> intentionally commented out to >

    printf("Program completed. Check with Valgrind for memory >
    return 0;
}
```

```
PS C:\Users\Gireesh> wsl
gireesh@Kannigantiii:/mnt/c/Users/Gireesh$ cd ~/oslab
gireesh@Kannigantiii:~/oslab$ nano memory_alloc.c
gireesh@Kannigantiii:~/oslab$ gcc -g memory_alloc.c -o memory_a
lloc
valgrind --leak-check=full ./memory_alloc
Command 'valgrind' not found, but can be installed with:
sudo apt install valgrind
gireesh@Kannigantiii:~/oslab$ sudo apt update
sudo apt install valgrind -y
[sudo: authenticate] Password:
Hit:1 http://archive.ubuntu.com/ubuntu resolute InRelease
Hit:2 http://security.ubuntu.com/ubuntu resolute-security InRel
ease
Hit:3 http://archive.ubuntu.com/ubuntu resolute-updates InRelea
se
Hit:4 http://archive.ubuntu.com/ubuntu resolute-backports InRel
ease
128 packages can be upgraded. Run 'apt list --upgradable' to se
e them.
Installing:
  valgrind
```

```
gireesh@Kannigantiii:~/oslab$ valgrind --leak-check=full ./memo
ry_alloc
==3494== Memcheck, a memory error detector
==3494== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Sewa
rd et al.
==3494== Using Valgrind-3.26.0 and LibVEX; rerun with -h for co
pyright info
==3494== Command: ./memory_alloc
==3494==
=== 1. malloc() demonstration ===
arr_malloc[0] = 10
arr_malloc[1] = 20
arr_malloc[2] = 30
arr_malloc[3] = 40
arr_malloc[4] = 50

=== 2. calloc() demonstration ===
arr_calloc[0] = 0 (initialized to zero)
arr_calloc[1] = 0 (initialized to zero)
arr_calloc[2] = 0 (initialized to zero)
arr_calloc[3] = 0 (initialized to zero)
arr_calloc[4] = 0 (initialized to zero)

=== 3. realloc() demonstration ===
After realloc (size 10):
arr_malloc[0] = 10
arr_malloc[1] = 20
arr_malloc[2] = 30
arr_malloc[3] = 40
arr_malloc[4] = 50
arr_malloc[5] = 60
arr_malloc[6] = 70
arr_malloc[7] = 80
arr_malloc[8] = 90
arr_malloc[9] = 100
```

```
=== 4. free() demonstration (Intentional Leak for Valgrind Test
) ===
Program completed. Check with Valgrind for memory leaks.
==3494==
==3494== HEAP SUMMARY:
==3494==     in use at exit: 400 bytes in 1 blocks
==3494==   total heap usage: 5 allocs, 4 frees, 1,504 bytes all
ocated
==3494==
==3494== 400 bytes in 1 blocks are definitely lost in loss reco
rd 1 of 1
==3494==    at 0x4850858: malloc (vg_replace_malloc.c:447)
==3494==    by 0x40014A5: main (memory_alloc.c:58)
==3494==
==3494== LEAK SUMMARY:
==3494==    definitely lost: 400 bytes in 1 blocks
==3494==    indirectly lost: 0 bytes in 0 blocks
==3494==      possibly lost: 0 bytes in 0 blocks
==3494==    still reachable: 0 bytes in 0 blocks
==3494==         suppressed: 0 bytes in 0 blocks
==3494==
==3494== For lists of detected and suppressed errors, rerun wit
h: -s
==3494== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0
 from 0)
```